

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа

**“Д35: Реализация межсервисного взаимодействия
посредством очередей сообщений”**

Выполнил:

Данилова Анастасия

Группа

К3339

Проверил:

Добряков Д. И.

**Санкт-Петербург
2026 г.**

Задача

1. Подключить и настроить rabbitMQ/kafka;
2. Реализовать межсервисное взаимодействие посредством rabbitMQ/kafka.

Выполнение

Использован RabbitMQ:

Клиент Go: github.com/rabbitmq/amqp091-go.

Топология RabbitMQ

Exchange

Параметр	Значение
Имя	ental.events
Тип	topic
Durable	да

Объявление: `shared/rabbitmq/topology.go` → `DeclareTopology()`.

События (routing keys)

Routing key	Publisher	Назначение
user.created	auth-service	Уведомление о регистрации
user.deleted	auth-service	Soft delete пользователя
property.created	property-service	Создание объявления
property.updated	property-service	Удаление объявления
property.deleted	property-service	

rental.created	rental-service	Новая заявка на аренду
rental.status_changed	rental-service	Смена статуса
rental.completed	rental-service	Завершение аренды
chat.created	chat-service	Новый чат
message.sent	chat-service	Новое сообщение

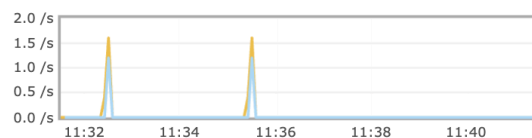
Payload описаны в shared/dto/events/events.go (JSON).

RabbitMQ Management → Exchanges → rental.events, список bindings.

Exchange: rental.events

Overview

Message rates **last ten minutes** ?



Publish (In) 0.00/s
Publish (Out) 0.00/s

Details

Type topic
Features durable: true
Policy

Bindings

This exchange



To	Routing key	Arguments	
chat-service.events	rental.created		Unbind
chat-service.events	user.deleted		Unbind
property.rental-events	rental.completed		Unbind
property.rental-events	rental.created		Unbind
property.user-deleted	user.deleted		Unbind
rental-service.user-deleted	user.deleted		Unbind

Демонстрирует созданный topic exchange.

Очереди и подписчики

Таблица consumers

Очередь	Binding keys	Сервис	Обработчик
property.user-deleted	user.deleted	property	Деактивация объявлений owner (is_available=false)
property.rental-events	rental.created, rental.completed	property	Занятость объекта
rental.user-deleted	user.deleted	rental	Отмена PENDING/ACTIVE аренд
chat.user-deleted	user.deleted	chat	is_archived=true на чатах
chat.rental-events	rental.created	chat	Авто-создание чата tenant↔landlord

Реализация:

- property: internal/consumers/user_deleted.go, rental_events.go
- rental: internal/consumer/user_deleted.go
- chat: internal/events/consumer.go

RabbitMQ Management → Queues — все очереди property.*, rental.*, chat.*.

Queues

▼ All queues (4)

Page 1 of 1 - Filter: ☐ Regex ? Displaying 4 items , page size

Overview									Messages					Message l
Virtual host	Name	Type	Features	Features	Policy	Consumers	Consumer capacity	State	Ready	Unacked	In Memory	Persistent	Total	Ready
/	chat-service.events	classic	D	D	?	1	100%	running	0	0	0	0	0	0 B
/	property.rental-events	classic	D	D	?	1	100%	running	0	0	0	0	0	0 B
/	property.user-deleted	classic	D	D	?	1	100%	running	0	0	0	0	0	0 B
/	rental-service.user-deleted	classic	D	D	?	1	100%	running	0	0	0	0	0	0 B

▶ Add new queue

Демонстрирует привязку очередей к exchange.

Публикация событий (publishers)

Общий publisher

shared/rabbitmq/publisher.go — Publish(ctx, routingKey, payload) с Content-Type: application/json, persistent delivery.

Примеры в коде

auth-service — после регистрации и soft delete:

```
go
publisher.Publish(ctx, events.UserCreated, events.UserCreatedPayload{...})
publisher.Publish(ctx, events.UserDeleted, events.UserDeletedPayload{...})
```

rental-service — после создания и завершения аренды:

```
go
publisher.Publish(ctx, events.RentalCreated, ...)
publisher.Publish(ctx, events.RentalCompleted, ...)
```

property-service — CRUD property → property.created / updated / deleted.

IDE: фрагмент rental_service.go с вызовом Publish.

```
func (s *PropertyService) CreateProperty(ctx context.Context, input CreatePropertyInput) (*models.Property, error) {
    if !input.PropertyType.IsValid() {
        return nil, ErrInvalidInput
    }
    if input.PricePerMonth < 0 {
        return nil, ErrInvalidInput
    }

    amenities, err := s.resolveAmenities(input.AmenityIDs)
    if err != nil {
        return nil, err
    }

    property := &models.Property{
        OwnerID:      input.OwnerID,
        Title:         input.Title,
        Description:   input.Description,
        PropertyType:  input.PropertyType,
        City:         input.City,
        Address:       input.Address,
        PricePerMonth: input.PricePerMonth,
        IsVerified:    false,
        IsAvailable:   true,
        Amenities:     amenities,
    }

    if err := s.Properties.Create(property); err != nil {
        return nil, err
    }
    if err := s.Properties.Reload(property); err != nil {
        return nil, err
    }
    _ = s.Publisher.Created(ctx, property.ID, property.OwnerID)
    return property, nil
}
```

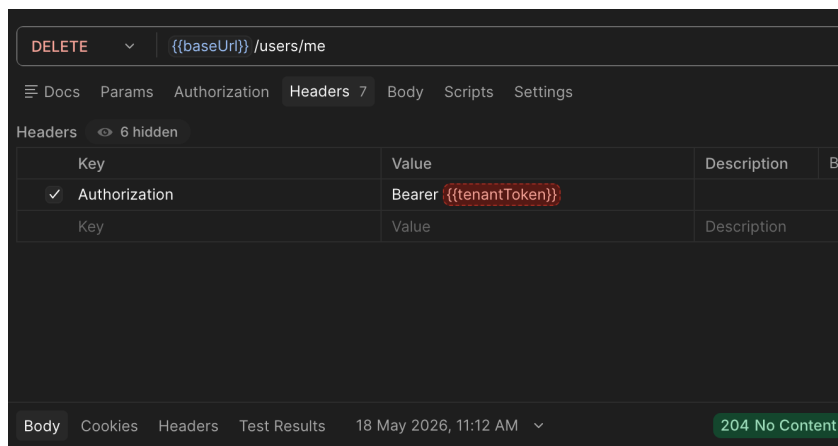
Демонстрирует: публикацию из бизнес-логики сервиса.

Сценарии использования (use cases)

Удаление пользователя (user.deleted)

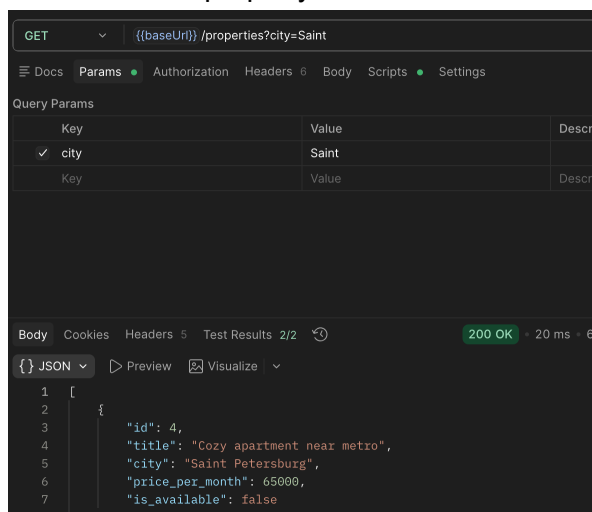
1. DELETE /api/users/me (auth) → soft delete, публикация события.
2. property — все properties с owner_id = user_id → is_available = false.
3. rental — аренды со статусом PENDING/ACTIVE для tenant/landlord → CANCELLED.
4. chat — чаты пользователя → is_archived = true.

Postman: Delete User — ответ 204.



Демонстрирует триггер цепочки событий.

Postman: GET property после delete use – список пуст.



Подключение в Docker Compose

Сервис rabbitmq в deploy/docker-compose.yml:

- AMQP: 5672
- Management: 15672
- Healthcheck: rabbitmq-diagnostics ping

Все микросервисы получают:

env

RABBITMQ_URL=amqp://guest:guest@rabbitmq:5672/

Фрагмент docker-compose с сервисом rabbitmq и env у auth-service.

```
Run Service
rabbitmq:
  image: rabbitmq:3.13-management-alpine
  hostname: rabbitmq
  volumes:
    - rabbitmq_data:/var/lib/rabbitmq
  ports:
    - "5672:5672"
    - "15672:15672"
  networks:
    - rental-net
  healthcheck:
    test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5
```

Демонстрирует инфраструктуру брокера в общем стеке.

docker compose logs property-service

```
api-gateway-1 | 192.168.65.1 - - [06/Jun/2026:08:35:29 +0000] "GET /api/rentals?role=tenant HTTP/1.1" 200 188 "-" "PostmanRuntime/7.54.0"
rental-service-1 | [GIN] 2026/06/06 - 08:35:29 | 200 | 2.828887ms | 192.168.65.1 | GET | "/rentals?role=tenant"
property-service-1 | [GIN] 2026/06/06 - 08:35:29 | 200 | 7.391419ms | 172.18.0.6 | GET | "/properties/3"
rental-service-1 | [GIN] 2026/06/06 - 08:35:29 | 200 | 16.22089ms | 192.168.65.1 | PATCH | "/rentals/3/complete"
api-gateway-1 | 192.168.65.1 - - [06/Jun/2026:08:35:29 +0000] "PATCH /api/rentals/3/complete HTTP/1.1" 200 529 "-" "PostmanRuntime/7.54.0"
property-service-1 | 2026/06/06 08:35:29 property_id=3 marked available (rental.completed)
chat-service-1 | [GIN] 2026/06/06 - 08:35:29 | 200 | 5.048925ms | 192.168.65.1 | GET | "/chats"
api-gateway-1 | 192.168.65.1 - - [06/Jun/2026:08:35:29 +0000] "GET /api/chats HTTP/1.1" 200 196 "-" "PostmanRuntime/7.54.0"
api-gateway-1 | 192.168.65.1 - - [06/Jun/2026:08:35:29 +0000] "GET /api/chats/3/messages?limit=50&offset=0 HTTP/1.1" 200 191 "-" "PostmanRuntime/7.54.0"
chat-service-1 | [GIN] 2026/06/06 - 08:35:29 | 200 | 4.263959ms | 192.168.65.1 | GET | "/chats/3/messages?limit=50&offset=0"
api-gateway-1 | 192.168.65.1 - - [06/Jun/2026:08:35:29 +0000] "PATCH /api/chats/3/read HTTP/1.1" 200 37 "-" "PostmanRuntime/7.54.0"
property-service-1 | 2026/06/06 08:35:29 deactivated properties for deleted user_id=8
auth-service-1 | [GIN] 2026/06/06 - 08:35:29 | 204 | 31.811269ms | 192.168.65.1 | DELETE | "/users/me"
api-gateway-1 | 192.168.65.1 - - [06/Jun/2026:08:35:29 +0000] "DELETE /api/users/me HTTP/1.1" 204 0 "-" "PostmanRuntime/7.54.0"
rental-service-1 | 2026/06/06 08:35:29 cancelled rentals for deleted user_id=8
chat-service-1 | 2026/06/06 08:35:29 archived chats for deleted user_id=8
rabbitmq-1 | 2026-06-06 08:35:44.401210+00:00 [info] <0.1898.0> Waiting for Mnesia tables for 30000 ms, 9 retries left
rabbitmq-1 | 2026-06-06 08:35:44.401623+00:00 [info] <0.1898.0> Successfully synced tables from a peer
postgres-1 | 2026-06-06 08:35:45.562 UTC [91] LOG: checkpoint starting: time
rabbitmq-1 | 2026-06-06 08:35:46.101890+00:00 [info] <0.1907.0> Waiting for Mnesia tables for 30000 ms, 9 retries left
rabbitmq-1 | 2026-06-06 08:35:46.102329+00:00 [info] <0.1907.0> Successfully synced tables from a peer
postgres-1 | 2026-06-06 08:35:49.198 UTC [91] LOG: checkpoint complete: wrote 34 buffers (0.2%); 0 WAL file(s) added, 0 removed, 0 recycle
d: writes=3.574 s, sync=0.024 s, total=3.636 s; sync files=34, longest=0.005 s, average=0.001 s; distance=20 kB, estimate=842 kB; lsn=0/26B1A80, redo
lsn=0/26B1A88
rabbitmq-1 | 2026-06-06 08:36:30.284707+00:00 [info] <0.1955.0> Waiting for Mnesia tables for 30000 ms, 9 retries left
rabbitmq-1 | 2026-06-06 08:36:30.285437+00:00 [info] <0.1955.0> Successfully synced tables from a peer
rabbitmq-1 | 2026-06-06 08:36:31.281237+00:00 [info] <0.1955.0> Successfully synced tables from a peer
```

Демонстрирует работа consumer в контейнере.

Выводы

1. RabbitMQ подключён через Docker Compose, доступен всем микросервисам по имени rabbitmq.
2. Реализован topic exchange rental.events и 10 типов доменных событий.
3. Consumers в property, rental и chat обрабатывают user.deleted, rental.created, rental.completed согласно ТЗ.
4. Publishers интегрированы в слой services; payload типизированы в shared/dto/events.
5. Асинхронное взаимодействие дополняет синхронные internal HTTP вызовы и заменяет каскадные FK из монолита.